Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное учреждение высшего образования

«Пермский национальный исследовательский политехнический университет»

Электротехнический факультет

Выпускающая кафедра: Информационные технологии и автоматизированные системы (ИТАС)

Направление подготовки: 09.03.04 Программная инженерия

ОТЧЕТ

Лабораторная работа №1

«Задача коммивояжера.»

По дисциплине «Основы алгоритмизации и программирования»

Вариант 9

Выполнил: студент группы РИС-25-2Б

Ильинов Фёдор Михайлович

Принял: Доц. Полякова О. А.

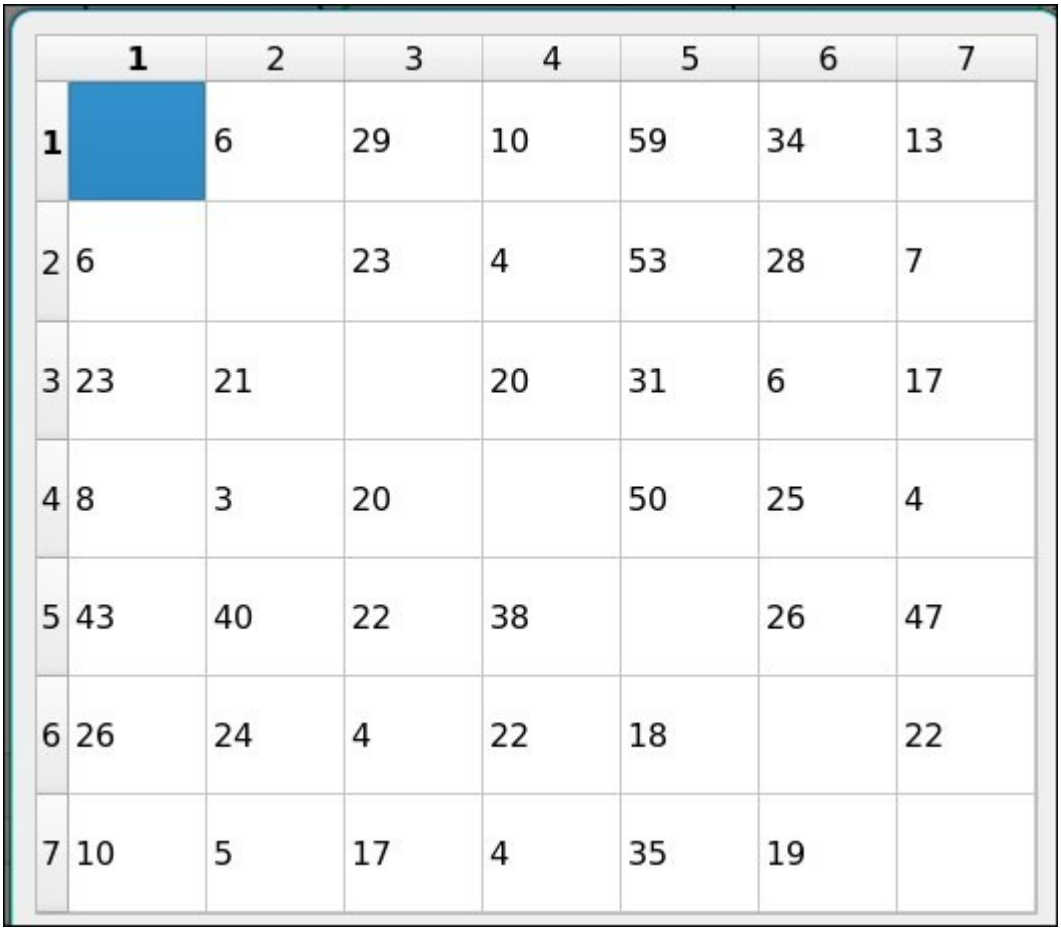
Пермь 2026

ПОСТАНОВКА ЗАДАЧИ

Задача коммивояжера формулируется следующим образом: дан полный взвешенный граф, где вершины представляют города, а веса рёбер — расстояния (или стоимость проезда) между ними. Требуется найти гамильтонов цикл минимального веса, то есть маршрут, проходящий через каждую вершину ровно один раз и возвращающийся в исходную.
Входными данными программы является матрица смежности, содержащая веса рёбер. Если ребро отсутствует, используется значение бесконечности (или достаточно большое число).

Метод ветвей и границ.
Суть метода заключается в последовательном разбиении множества допустимых решений на подмножества (ветвление) и вычислении для каждого подмножества оценки снизу (границы).
В данной реализации алгоритм работает по следующему принципу:
Ветвление: на каждом шаге алгоритм пытается добавить в текущий путь следующую вершину.
Оценка: вычисляется нижняя оценка стоимости маршрута. В данной работе используется эвристическая оценка: к текущей стоимости пройденного пути прибавляется произведение количества оставшихся вершин на минимальный вес ребра во всём графе.
Отсечение: если вычисленная нижняя оценка превышает стоимость уже найденного лучшего решения (верхнюю границу), данная ветвь поиска прекращается, так как она не может привести к оптимальному решению.
Рекурсия: Процесс повторяется до тех пор, пока не будет построен полный маршрут или не будут отсечены все неперспективные ветви.

Вариант 9



	1	2	3	4	5	6	7
1		6	29	10	59	34	13
2	6		23	4	53	28	7
3	23	21		20	31	6	17
4	8	3	20		50	25	4
5	43	40	22	38		26	47
6	26	24	4	22	18		22
7	10	5	17	4	35	19	

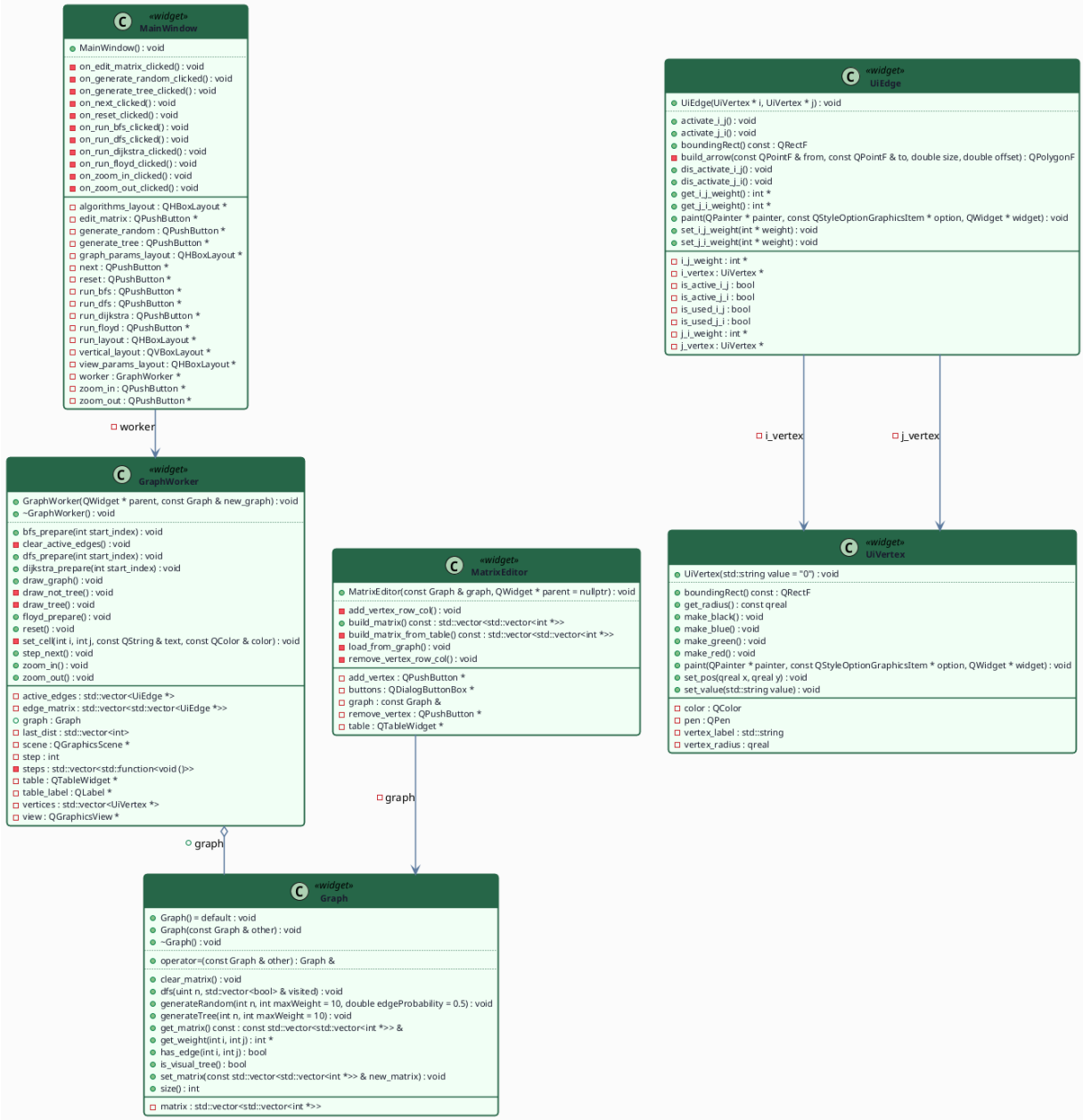
АНАЛИЗ.

Graph, UiVertex, UiEdge, MatrixEditor — см. работу “Графы”

GraphWorker — отрисовщик шагов алгоритма.

MainWindow — кнопки: Edit Matrix, Generate Random/Tree, Run BnB, Zoom In/Out, Reset, Next Step. Next Step запускает QTimer с interval=0, который вызывает сигнал step_next() пока шаги не кончатся.

UML-ДИАГРАММА



КОД UiVertex.h

```
//
// Created by localuser on 5/12/26.
//

#ifndef QTGRAPHVISUALIZATION_UIVERTEX_H
#define QTGRAPHVISUALIZATION_UIVERTEX_H
#include <QGraphicsScene>
#include <QGraphicsItem>
#include <QPainter>
```

```
class UiVertex : public QGraphicsItem {
    std::string vertex_label = "0";
    qreal vertex_radius = 30;

    QColor color = Qt::black;
    QPen pen = QPen(Qt::black, 3);
public:
    UiVertex(std::string value = "0");

    void set_pos(qreal x, qreal y);
    void set_value(std::string value);
    const qreal get_radius();

    void make_red();
    void make_green();
    void make_blue();
    void make_black();
    QRectF boundingRect() const override;

    void paint(QPainter *painter,
            const QStyleOptionGraphicsItem *option,
            QWidget *widget) override;
};

#endif //QTGRAPHVISUALIZATION_UIVERTEX_H
```

КОД UiVertex.cpp

```cpp
//
// Created by localuser on 5/12/26.
//

#include "UiVertex.h"

UiVertex::UiVertex(std::string value) {
    vertex_label = value;
}

QRectF UiVertex::boundingRect() const {
    return QRectF(-vertex_radius, -vertex_radius,
                  vertex_radius * 2,
                  vertex_radius * 2);
}

void UiVertex::paint(QPainter *painter,
                     const QStyleOptionGraphicsItem *,
                     QWidget *) {

    painter->setPen(pen);

    painter->setBrush(Qt::white);
    painter->drawEllipse(boundingRect());

    painter->setPen(Qt::black);
    painter->drawText(boundingRect(),
                      Qt::AlignCenter,
                      QString::fromStdString(vertex_label));
}

void UiVertex::set_pos(qreal x, qreal y) {
    setPos(x, y);
}

void UiVertex::set_value(std::string value) {
    vertex_label = value;
    update();
}

const qreal UiVertex::get_radius() {
    return vertex_radius;
}
```

```
void UiVertex::make_red() {
    pen = QPen(Qt::red, 5);
    update();
}
void UiVertex::make_blue() {
    pen = QPen(Qt::blue, 5);
    update();
}
void UiVertex::make_green() {
    pen = QPen(Qt::green, 3 );
    update();
}

void UiVertex::make_black() {
    pen = QPen(Qt::black, 3);
    update();
}
```

КОД UiEdge.h

```
#ifndef QTGRAPHVISUALIZATION_UIEDGE_H
#define QTGRAPHVISUALIZATION_UIEDGE_H
#include <QGraphicsItem>
#include "UiVertex.h"

class UiEdge : public QGraphicsItem {
    UiVertex *i_vertex;
    UiVertex *j_vertex;
    int* i_j_weight = nullptr;
    int* j_i_weight = nullptr;
    bool is_active_i_j = false;
    bool is_active_j_i = false;
    bool is_used_i_j = false;
    bool is_used_j_i = false;

    QPolygonF build_arrow(const QPointF &from, const QPointF &to, double size, double offset);

public:
    enum class DirColor { None, Included, Excluded, Current };

    UiEdge(UiVertex *i, UiVertex *j);

    QRectF boundingRect() const override;

    void paint(QPainter *painter,
            const QStyleOptionGraphicsItem *option,
            QWidget *widget) override;
    void set_i_j_weight(int* weight);
    void set_j_i_weight(int* weight);

    void activate_i_j();
    void dis_activate_i_j();

    void activate_j_i();
    void dis_activate_j_i();

    void reset_colors();
    void set_i_j_color(DirColor c);
    void set_j_i_color(DirColor c);

    int* get_i_j_weight();;
    int* get_j_i_weight();;
```

```
private:
    DirColor i_j_color = DirColor::None;
    DirColor j_i_color = DirColor::None;
};

#endif
```

КОД UiEdge.cpp

```
#include "UiEdge.h"

#include "UiVertex.h"

UiEdge::UiEdge(UiVertex *i, UiVertex *j) {
    i_vertex = i;
    j_vertex = j;
}

QRectF UiEdge::boundingRect() const {
    auto aleft = std::min(i_vertex->pos().x(), j_vertex->pos().x()) - i_vertex->get_radius();
    auto atop = std::min(i_vertex->pos().y(), j_vertex->pos().y()) - i_vertex->get_radius();
    auto awidth = abs(i_vertex->pos().x() - j_vertex->pos().x()) + 2 * i_vertex->get_radius();
    auto aheight = abs(i_vertex->pos().y() - j_vertex->pos().y()) + 2 * i_vertex->get_radius();

    return QRectF(aleft, atop, awidth, aheight);
}

static QColor color_for_dir(UiEdge::DirColor c) {
    switch (c) {
        case UiEdge::DirColor::Included: return Qt::green;
        case UiEdge::DirColor::Excluded: return Qt::blue;
        case UiEdge::DirColor::Current:  return Qt::red;
        default: return Qt::black;
    }
}

void UiEdge::paint(QPainter *painter,
                   const QStyleOptionGraphicsItem *,
                   QWidget *) {
    if (i_vertex->pos() == j_vertex->pos()) return;

    DirColor effective = DirColor::None;
    if (i_j_color == DirColor::Current || j_i_color == DirColor::Current)
        effective = DirColor::Current;
    else if (i_j_color == DirColor::Included || j_i_color == DirColor::Included)
        effective = DirColor::Included;
```

```
else if (i_j_color == DirColor::Excluded || j_i_color == DirColor::Excluded)
    effective = DirColor::Excluded;

QColor line_color = Qt::black;
int line_width = 3;
if (effective == DirColor::Current || is_active_i_j || is_active_j_i) {
    line_color = Qt::red;
    line_width = 5;
} else if (effective == DirColor::Included) {
    line_color = Qt::green;
    line_width = 5;
} else if (effective == DirColor::Excluded) {
    line_color = Qt::blue;
    line_width = 3;
}
painter->setPen(QPen(line_color, line_width));
painter->drawLine(i_vertex->pos(), j_vertex->pos());

auto pen_for_dir = [&](DirColor c, bool active) {
    if (active) return QPen(Qt::red, 5);
    return QPen(color_for_dir(c), 3);
};

if (i_j_weight != nullptr) {
    QPolygonF arrow = build_arrow(i_vertex->pos(), j_vertex->pos(),
                                  i_vertex->get_radius() * 2, i_vertex->get_radius());
    painter->setPen(pen_for_dir(i_j_color, is_active_i_j));
    painter->setBrush(QBrush(Qt::white));
    painter->drawPolygon(arrow);

    QPointF centroid = (arrow[0] + arrow[1] + arrow[2]) / 3.0;
    painter->setPen(QPen(Qt::black));
    QString text = QString::number(*this->i_j_weight);
    if (*i_j_weight == INT_MAX) text = "+inf";
    if (*i_j_weight == INT_MIN) text = "-inf";
    painter->drawText((int)(centroid.x() - 20), (int)(centroid.y() - 10),
                      40, 20, Qt::AlignCenter, text);
}

if (j_i_weight != nullptr) {
    QPolygonF arrow = build_arrow(j_vertex->pos(), i_vertex->pos(),
                                  i_vertex->get_radius() * 2, i_vertex->get_radius());
    painter->setPen(pen_for_dir(j_i_color, is_active_j_i));
    painter->setBrush(QBrush(Qt::white));
    painter->drawPolygon(arrow);
```

```
        QPointF centroid = (arrow[0] + arrow[1] + arrow[2]) / 3.0;
        painter->setPen(QPen(Qt::black));
        QString text = QString::number(*this->j_i_weight);
        if (*j_i_weight == INT_MAX) text = "+inf";
        if (*j_i_weight == INT_MIN) text = "-inf";
        painter->drawText((int)(centroid.x() - 20), (int)(centroid.y() - 10),
                          40, 20, Qt::AlignCenter, text);
    }
}

QPolygonF UiEdge::build_arrow(const QPointF &from, const QPointF &to,
                              double size, double offset) {
    QPointF dir = to - from;
    double len = std::sqrt(dir.x() * dir.x() + dir.y() * dir.y());
    if (len == 0) return QPolygonF();

    QPointF unit = dir / len;
    QPointF perp(-unit.y(), unit.x());

    double height = size * std::sqrt(3.0) / 2.0;

    QPointF tip = to - unit * offset;
    QPointF base1 = tip - unit * height + perp * (size / 2.0);
    QPointF base2 = tip - unit * height - perp * (size / 2.0);
    QPolygonF pol;
    pol.resize(3);
    pol[0] = tip;
    pol[1] = base1;
    pol[2] = base2;
    return pol;
}

void UiEdge::set_i_j_weight(int *weight) { i_j_weight = weight; update(); }
void UiEdge::set_j_i_weight(int *weight) { j_i_weight = weight; update(); }

void UiEdge::activate_i_j() { is_active_i_j = true; update(); }
void UiEdge::dis_activate_i_j() { is_active_i_j = false; update(); }
void UiEdge::activate_j_i() { is_active_j_i = true; update(); }
void UiEdge::dis_activate_j_i() { is_active_j_i = false; update(); }

void UiEdge::reset_colors() { i_j_color = DirColor::None; j_i_color = DirColor::None; update(); }
void UiEdge::set_i_j_color(DirColor c) { i_j_color = c; update(); }
void UiEdge::set_j_i_color(DirColor c) { j_i_color = c; update(); }
```

```
int *UiEdge::get_i_j_weight() { return i_j_weight; }
int *UiEdge::get_j_i_weight() { return j_i_weight; }
```

КОД MatrixEditor.h

```
//
// Created by localuser on 5/13/26.
//

#ifndef QTGRAPHVISUALIZATION_MATRIXEDITOR_H
#define QTGRAPHVISUALIZATION_MATRIXEDITOR_H

#include <QDialog>
#include <QTableWidget>
#include <QDialogButtonBox>
#include <QPushButton>
#include "Graph.h"

class MatrixEditor : public QDialog {
    Q_OBJECT

    const Graph &graph;
    QTableWidget *table;
    QPushButton *add_vertex;
    QPushButton *remove_vertex;
    QDialogButtonBox *buttons;

    void load_from_graph();
    void add_vertex_row_col();
    void remove_vertex_row_col();
    std::vector<std::vector<int *> > build_matrix_from_table() const;

public:
    MatrixEditor(const Graph &graph, QWidget *parent = nullptr);
    std::vector<std::vector<int *> > build_matrix() const;
};

#endif //QTGRAPHVISUALIZATION_MATRIXEDITOR_H
```

КОД MainWindow.h

```
#ifndef QTGRAPHVISUALIZATION_MAINWINDOW_H
#define QTGRAPHVISUALIZATION_MAINWINDOW_H

#include <QHBoxLayout>
#include <QGraphicsView>
#include <QGraphicsScene>
#include <QPushButton>
#include <QDialog>
#include <QSpinBox>
#include <QDialogButtonBox>
#include <QTimer>
#include "GraphWorker.h"
#include "MatrixEditor.h"

class MainWindow : public QWidget {
    Q_OBJECT
    QVBoxLayout *vertical_layout;
    QHBoxLayout *graph_params_layout;
    QHBoxLayout *run_layout;

    QPushButton *edit_matrix;
    QPushButton *generate_random;
    QPushButton *generate_tree;

    QPushButton *run_bnb;
    QPushButton *reset;
    QPushButton *next;
    QPushButton *zoom_in;
    QPushButton *zoom_out;

    GraphWorker *worker;
    QTimer *anim_timer = nullptr;

public:
    MainWindow();

private slots:
    void on_edit_matrix_clicked();
    void on_generate_random_clicked();
    void on_generate_tree_clicked();
    void on_run_bnb_clicked();
    void on_reset_clicked();
    void on_next_clicked();
    void on_zoom_in_clicked();
    void on_zoom_out_clicked();
```

```
};

#endif
```

КОД MainWindow.cpp

```
#include "MainWindow.h"

#include "UiEdge.h"
#include <chrono>
#include <thread>

MainWindow::MainWindow() : QWidget(nullptr) {
    vertical_layout = new QVBoxLayout(this);

    Graph graph;
    worker = new GraphWorker(this);
    worker->graph = graph;
    worker->draw_graph();

    graph_params_layout = new QHBoxLayout;
    edit_matrix = new QPushButton("Edit matrix", this);
    generate_random = new QPushButton("Generate random graph", this);
    generate_tree = new QPushButton("Generate tree", this);

    graph_params_layout->addWidget(edit_matrix);
    graph_params_layout->addWidget(generate_random);
    graph_params_layout->addWidget(generate_tree);
    vertical_layout->addLayout(graph_params_layout);

    vertical_layout->addWidget(worker);

    run_layout = new QHBoxLayout;
    run_bnb = new QPushButton("Run BnB", this);
    zoom_in = new QPushButton("Zoom In", this);
    zoom_out = new QPushButton("Zoom Out", this);
    reset = new QPushButton("Reset", this);
    next = new QPushButton("Next Step", this);

    run_layout->addWidget(run_bnb);
    run_layout->addWidget(zoom_in);
    run_layout->addWidget(zoom_out);
    run_layout->addWidget(reset);
    run_layout->addWidget(next);
    vertical_layout->addLayout(run_layout);
```

```
    connect(edit_matrix, &QPushButton::clicked, this, &MainWindow::on_edit_matrix_clicked);
    connect(generate_random, &QPushButton::clicked, this,
&MainWindow::on_generate_random_clicked);
    connect(generate_tree, &QPushButton::clicked, this,
&MainWindow::on_generate_tree_clicked);
    connect(run_bnb, &QPushButton::clicked, this, &MainWindow::on_run_bnb_clicked);
    connect(zoom_in, &QPushButton::clicked, this, &MainWindow::on_zoom_in_clicked);
    connect(zoom_out, &QPushButton::clicked, this, &MainWindow::on_zoom_out_clicked);
    connect(next, &QPushButton::clicked, this, &MainWindow::on_next_clicked);
    connect(reset, &QPushButton::clicked, this, &MainWindow::on_reset_clicked);
}

void MainWindow::on_edit_matrix_clicked() {
    MatrixEditor editor(worker->graph, this);
    if (editor.exec() == QDialog::Accepted) {
        worker->graph.set_matrix(editor.build_matrix());
        worker->draw_graph();
    }
}

void MainWindow::on_generate_random_clicked() {
    QDialog dialog(this);
    dialog.setWindowTitle("Vertices amount");

    auto *layout = new QVBoxLayout(&dialog);
    auto *spin = new QSpinBox(&dialog);
    spin->setRange(1, 10);
    spin->setValue(4);

    auto *buttons = new QDialogButtonBox(QDialogButtonBox::Ok | QDialogButtonBox::Cancel,
&dialog);
    layout->addWidget(spin);
    layout->addWidget(buttons);

    connect(buttons, &QDialogButtonBox::accepted, &dialog, &QDialog::accept);
    connect(buttons, &QDialogButtonBox::rejected, &dialog, &QDialog::reject);

    if (dialog.exec() != QDialog::Accepted) return;

    worker->graph.generateRandom(spin->value(), 10, 1.0);
    worker->draw_graph();
}

void MainWindow::on_generate_tree_clicked() {
```

```
    QDialog dialog(this);
    dialog.setWindowTitle("Vertices amount");

    auto *layout = new QVBoxLayout(&dialog);
    auto *spin = new QSpinBox(&dialog);
    spin->setRange(1, 10);
    spin->setValue(4);

    auto *buttons = new QDialogButtonBox(QDialogButtonBox::Ok | QDialogButtonBox::Cancel,
&dialog);
    layout->addWidget(spin);
    layout->addWidget(buttons);

    connect(buttons, &QDialogButtonBox::accepted, &dialog, &QDialog::accept);
    connect(buttons, &QDialogButtonBox::rejected, &dialog, &QDialog::reject);

    if (dialog.exec() != QDialog::Accepted) return;

    worker->graph.generateTree(spin->value(), 10);
    worker->draw_graph();
}

void MainWindow::on_run_bnb_clicked() {
    int n = worker->graph.size();
    if (n == 0) return;

    worker->reset();
    worker->bnb_prepare();
}

void MainWindow::on_reset_clicked() {
    worker->reset();
}

void MainWindow::on_next_clicked() {
    if (anim_timer && anim_timer->isActive()) return;
    if (!worker->has_next()) return;
    if (!anim_timer) {
        anim_timer = new QTimer(this);
        connect(anim_timer, &QTimer::timeout, this, [this]() {
            worker->step_next();
            if (!worker->has_next())
                anim_timer->stop();
        });
    }
```

```
    anim_timer->start(0);
}

void MainWindow::on_zoom_in_clicked() {
    worker->zoom_in();
}

void MainWindow::on_zoom_out_clicked() {
    worker->zoom_out();
}
```

КОД main.cpp

```
#include <QApplication>
#include <QPushButton>
#include "MainWindow.h"

int main(int argc, char* argv[]) {
    QApplication a(argc, argv);
    MainWindow window;
    window.show();
    return QApplication::exec();
}
```

КОД GraphWorker.h

```
#include "MainWindow.h"

#include "UiEdge.h"
#include <chrono>
#include <thread>

MainWindow::MainWindow() : QWidget(nullptr) {
    vertical_layout = new QVBoxLayout(this);

    Graph graph;
    worker = new GraphWorker(this);
    worker->graph = graph;
    worker->draw_graph();

    graph_params_layout = new QHBoxLayout;
    edit_matrix = new QPushButton("Edit matrix", this);
    generate_random = new QPushButton("Generate random graph", this);
    generate_tree = new QPushButton("Generate tree", this);
```

```
    graph_params_layout->addWidget(edit_matrix);
    graph_params_layout->addWidget(generate_random);
    graph_params_layout->addWidget(generate_tree);
    vertical_layout->addLayout(graph_params_layout);

    vertical_layout->addWidget(worker);

    run_layout = new QHBoxLayout;
    run_bnb = new QPushButton("Run BnB", this);
    zoom_in = new QPushButton("Zoom In", this);
    zoom_out = new QPushButton("Zoom Out", this);
    reset = new QPushButton("Reset", this);
    next = new QPushButton("Next Step", this);

    run_layout->addWidget(run_bnb);
    run_layout->addWidget(zoom_in);
    run_layout->addWidget(zoom_out);
    run_layout->addWidget(reset);
    run_layout->addWidget(next);
    vertical_layout->addLayout(run_layout);

    connect(edit_matrix, &QPushButton::clicked, this, &MainWindow::on_edit_matrix_clicked);
    connect(generate_random, &QPushButton::clicked, this,
&MainWindow::on_generate_random_clicked);
    connect(generate_tree, &QPushButton::clicked, this,
&MainWindow::on_generate_tree_clicked);
    connect(run_bnb, &QPushButton::clicked, this, &MainWindow::on_run_bnb_clicked);
    connect(zoom_in, &QPushButton::clicked, this, &MainWindow::on_zoom_in_clicked);
    connect(zoom_out, &QPushButton::clicked, this, &MainWindow::on_zoom_out_clicked);
    connect(next, &QPushButton::clicked, this, &MainWindow::on_next_clicked);
    connect(reset, &QPushButton::clicked, this, &MainWindow::on_reset_clicked);
}

void MainWindow::on_edit_matrix_clicked() {
    MatrixEditor editor(worker->graph, this);
    if (editor.exec() == QDialog::Accepted) {
        worker->graph.set_matrix(editor.build_matrix());
        worker->draw_graph();
    }
}

void MainWindow::on_generate_random_clicked() {
    QDialog dialog(this);
    dialog.setWindowTitle("Vertices amount");
```

```
    auto *layout = new QVBoxLayout(&dialog);
    auto *spin = new QSpinBox(&dialog);
    spin->setRange(1, 10);
    spin->setValue(4);

    auto *buttons = new QDialogButtonBox(QDialogButtonBox::Ok | QDialogButtonBox::Cancel,
&dialog);
    layout->addWidget(spin);
    layout->addWidget(buttons);

    connect(buttons, &QDialogButtonBox::accepted, &dialog, &QDialog::accept);
    connect(buttons, &QDialogButtonBox::rejected, &dialog, &QDialog::reject);

    if (dialog.exec() != QDialog::Accepted) return;

    worker->graph.generateRandom(spin->value(), 10, 1.0);
    worker->draw_graph();
}

void MainWindow::on_generate_tree_clicked() {
    QDialog dialog(this);
    dialog.setWindowTitle("Vertices amount");

    auto *layout = new QVBoxLayout(&dialog);
    auto *spin = new QSpinBox(&dialog);
    spin->setRange(1, 10);
    spin->setValue(4);

    auto *buttons = new QDialogButtonBox(QDialogButtonBox::Ok | QDialogButtonBox::Cancel,
&dialog);
    layout->addWidget(spin);
    layout->addWidget(buttons);

    connect(buttons, &QDialogButtonBox::accepted, &dialog, &QDialog::accept);
    connect(buttons, &QDialogButtonBox::rejected, &dialog, &QDialog::reject);

    if (dialog.exec() != QDialog::Accepted) return;

    worker->graph.generateTree(spin->value(), 10);
    worker->draw_graph();
}

void MainWindow::on_run_bnb_clicked() {
    int n = worker->graph.size();
```

```
    if (n == 0) return;

    worker->reset();
    worker->bnb_prepare();
}

void MainWindow::on_reset_clicked() {
    worker->reset();
}

void MainWindow::on_next_clicked() {
    if (anim_timer && anim_timer->isActive()) return;
    if (!worker->has_next()) return;
    if (!anim_timer) {
        anim_timer = new QTimer(this);
        connect(anim_timer, &QTimer::timeout, this, [this]() {
            worker->step_next();
            if (!worker->has_next())
                anim_timer->stop();
        });
    }
    anim_timer->start(0);
}

void MainWindow::on_zoom_in_clicked() {
    worker->zoom_in();
}

void MainWindow::on_zoom_out_clicked() {
    worker->zoom_out();
}
```

КОД GraphWorker.cpp

```
#include "GraphWorker.h"

#include <QGraphicsView>
#include <QVBoxLayout>
#include <cmath>
#include <queue>
#include <algorithm>
#include <climits>

static const int BNB_INF = 1e9;
```

```
GraphWorker::GraphWorker(QWidget *parent) : QWidget(parent) {
    table = new QTableWidget(this);
    table->hide();
    table->setMaximumHeight(200);
    table_label = new QLabel(this);
    table_label->hide();

    scene = new QGraphicsScene(this);
    view = new QGraphicsView(scene, this);

    view->setRenderHint(QPainter::Antialiasing);
    view->setSizePolicy(QSizePolicy::Expanding, QSizePolicy::Expanding);
    view->setTransformationAnchor(QGraphicsView::AnchorUnderMouse);
    view->setInteractive(true);
    view->setDragMode(QGraphicsView::ScrollHandDrag);

    auto *layout = new QVBoxLayout(this);
    layout->setContentsMargins(0, 0, 0, 0);
    layout->addWidget(view);
    layout->addWidget(table_label);
    layout->addWidget(table);
    setLayout(layout);

    setSizePolicy(QSizePolicy::Expanding, QSizePolicy::Expanding);
}

GraphWorker::~GraphWorker() {
    for (auto *v : vertices) delete v;
    for (auto v : edge_matrix)
        for (auto e : v)
            if (e) delete e;
}

void GraphWorker::draw_not_tree() {
    scene->clear();
    vertices.clear();
    edge_matrix.clear();

    int n = graph.size();
    if (n == 0) return;

    const double radius_val = 500.0;
    const double step_angle = 2.0 * M_PI / n;
    QPointF center = view->mapToScene(view->viewport()->rect().center());
```

```
    for (int i = 0; i < n; i++) {
        double angle = step_angle * i;
        double x = center.x() + radius_val * std::cos(angle);
        double y = center.y() + radius_val * std::sin(angle);
        auto *v = new UiVertex(std::to_string(i + 1));
        v->set_pos(x, y);
        vertices.push_back(v);
    }

    edge_matrix.assign(n, std::vector<UiEdge *>(n, nullptr));

    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (graph.has_edge(i, j) || graph.has_edge(j, i)) {
                auto *e = new UiEdge(vertices[i], vertices[j]);
                if (graph.has_edge(i, j)) {
                    e->set_i_j_weight(graph.get_weight(i, j));
                    edge_matrix[i][j] = e;
                }
                if (graph.has_edge(j, i)) {
                    e->set_j_i_weight(graph.get_weight(j, i));
                    edge_matrix[j][i] = e;
                }
            }
        }
    }

    for (auto v : edge_matrix)
        for (auto ui_edge : v)
            if (ui_edge)
                scene->addItem(ui_edge);
    for (auto v : vertices)
        scene->addItem(v);
}

void GraphWorker::draw_graph() {
    draw_not_tree();
}

void GraphWorker::zoom_in() {
    view->setTransformationAnchor(QGraphicsView::AnchorViewCenter);
    view->scale(1.2, 1.2);
}

void GraphWorker::zoom_out() {
```

```
    view->setTransformationAnchor(QGraphicsView::AnchorViewCenter);
    view->scale(1.0 / 1.2, 1.0 / 1.2);
}

void GraphWorker::set_cell(int i, int j, const QString &text, const QColor &color) {
    auto *item = table->item(i, j);
    if (!item) {
        item = new QTableWidgetItem();
        table->setItem(i, j, item);
    }
    item->setText(text);
    item->setBackground(color);
    item->setTextAlignment(Qt::AlignCenter);
}

void GraphWorker::clear_edge_highlights() {
    for (auto v : edge_matrix)
        for (auto e : v)
            if (e) {
                e->reset_colors();
                e->update();
            }
}

void GraphWorker::step_next() {
    if (step >= static_cast<int>(steps.size())) return;
    steps[step]();
    step++;
}

void GraphWorker::reset() {
    steps.clear();
    step = 0;
    clear_edge_highlights();
    for (int i = 0; i < static_cast<int>(vertices.size()); i++) {
        vertices[i]->make_black();
        vertices[i]->set_value(std::to_string(i + 1));
        vertices[i]->update();
    }
    for (auto v : edge_matrix)
        for (auto e : v)
            if (e) {
                e->dis_activate_i_j();
                e->dis_activate_j_i();
                e->update();
```

```
        }
    table->hide();
    table_label->hide();
}

void GraphWorker::bnb_prepare() {
    steps.clear();
    step = 0;

    int n = graph.size();
    if (n == 0) return;

    // Build cost matrix from graph
    std::vector<std::vector<int>> mat(n, std::vector<int>(n, BNB_INF));
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            if (i != j && graph.has_edge(i, j))
                mat[i][j] = *graph.get_weight(i, j);

    // Setup table
    table->show();
    table_label->show();
    table->setRowCount(n);
    table->setColumnCount(n);
    for (int i = 0; i < n; i++) {
        table->setHorizontalHeaderItem(i, new QTableWidgetItem(QString::number(i + 1)));
        table->setVerticalHeaderItem(i, new QTableWidgetItem(QString::number(i + 1)));
        table->setColumnWidth(i, 50);
        table->setRowHeight(i, 30);
    }

    // Helper: convert working m×m matrix to full n×n display matrix
    auto build_full = [n](const std::vector<std::vector<int>> &small,
                          const std::vector<int> &rows,
                          const std::vector<int> &cols) {
        std::vector<std::vector<int>> full(n, std::vector<int>(n, BNB_INF));
        for (size_t i = 0; i < small.size() && i < rows.size(); i++)
            for (size_t j = 0; j < small[i].size() && j < cols.size(); j++)
                if (rows[i] < n && cols[j] < n)
                    full[rows[i]][cols[j]] = small[i][j];
        return full;
    };

    // Helper: push a visualization step
    auto push_step = [this, n](const std::vector<std::vector<int>> &matrix,
```

```
                    int bound, int best,
                    const std::vector<std::pair<int, int>> &included,
                    const std::vector<std::pair<int, int>> &excluded,
                    int branch_i, int branch_j,
                    const std::string &msg) {
    steps.push_back([this, n, matrix, bound, best, included, excluded, branch_i, branch_j, msg]
() {
      for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++) {
          QString val;
          QColor bg = Qt::white;
          if (matrix[i][j] >= BNB_INF / 2)
            val = "inf";
          else
            val = QString::number(matrix[i][j]);
          set_cell(i, j, val, bg);
        }

      QString label = QString::fromStdString(msg);
      if (bound < BNB_INF / 2)
        label += QString("  LB: %1").arg(bound);
      if (best < BNB_INF / 2)
        label += QString("  Best: %1").arg(best);
      table_label->setText(label);

      clear_edge_highlights();
      auto color_arrow = [](UiEdge *e, int from, int to, UiEdge::DirColor c) {
        if (from < to) e->set_i_j_color(c);
        else           e->set_j_i_color(c);
      };
      for (auto &p : included)
        if (edge_matrix[p.first][p.second])
          color_arrow(edge_matrix[p.first][p.second], p.first, p.second,
                      UiEdge::DirColor::Included);
      for (auto &p : excluded)
        if (edge_matrix[p.first][p.second])
          color_arrow(edge_matrix[p.first][p.second], p.first, p.second,
                      UiEdge::DirColor::Excluded);
      if (branch_i >= 0 && branch_j >= 0 && edge_matrix[branch_i][branch_j])
        color_arrow(edge_matrix[branch_i][branch_j], branch_i, branch_j,
                    UiEdge::DirColor::Current);
    });
  };

  // Initial best = INF, best_path empty
```

```
int best = BNB_INF;
std::vector<std::pair<int, int>> best_path;

push_step(mat, 0, best, {}, {}, -1, -1, "Initial matrix");

// Recursive BnB (Little's algorithm)
std::function<void(std::vector<std::vector<int>>, int,
                   std::vector<std::pair<int, int>>,
                   std::vector<std::pair<int, int>>,
                   std::vector<int>, std::vector<int>)>
  bnb_rec;

bnb_rec = [&](std::vector<std::vector<int>> cmat, int bound,
              std::vector<std::pair<int, int>> included,
              std::vector<std::pair<int, int>> excluded,
              std::vector<int> row_cities,
              std::vector<int> col_cities) {
  int m = cmat.size();

  // Reduce rows
  for (int i = 0; i < m; i++) {
    int row_min = BNB_INF;
    for (int j = 0; j < m; j++)
      if (cmat[i][j] < row_min) row_min = cmat[i][j];
    if (row_min >= BNB_INF / 2) return; // impossible
    if (row_min > 0) {
      for (int j = 0; j < m; j++)
        if (cmat[i][j] < BNB_INF / 2)
          cmat[i][j] -= row_min;
      bound += row_min;
      push_step(build_full(cmat, row_cities, col_cities), bound, best,
                included, excluded, -1, -1,
                "Row reduce city " + std::to_string(row_cities[i] + 1) +
                  ", sub " + std::to_string(row_min));
    }
  }

  // Reduce columns
  for (int j = 0; j < m; j++) {
    int col_min = BNB_INF;
    for (int i = 0; i < m; i++)
      if (cmat[i][j] < col_min) col_min = cmat[i][j];
    if (col_min >= BNB_INF / 2) return;
    if (col_min > 0) {
      for (int i = 0; i < m; i++)
```

```
            if (cmat[i][j] < BNB_INF / 2)
                cmat[i][j] -= col_min;
        bound += col_min;
        push_step(build_full(cmat, row_cities, col_cities), bound, best,
                  included, excluded, -1, -1,
                  "Column reduce city " + std::to_string(col_cities[j] + 1) +
                    ", sub " + std::to_string(col_min));
    }
}

// Prune
if (bound >= best) {
    push_step(build_full(cmat, row_cities, col_cities), bound, best,
              included, excluded, -1, -1, "Prune (bound >= best)");
    return;
}

// Base case: 2x2 matrix → complete tour (try all 2 matchings)
if (m == 2) {
    static const int matchings[2][2][2] = {{{0,1},{1,0}}, {{0,0},{1,1}}};
    bool found_tour = false;
    for (auto &matching : matchings) {
        int u = matching[0][0], v = matching[0][1];
        int x = matching[1][0], y = matching[1][1];
        if (cmat[u][v] < BNB_INF / 2 && cmat[x][y] < BNB_INF / 2) {
            auto tour = included;
            tour.emplace_back(row_cities[u], col_cities[v]);
            tour.emplace_back(row_cities[x], col_cities[y]);
            if (bound < best) {
                best = bound;
                best_path = tour;
            }
            push_step(build_full(cmat, row_cities, col_cities), bound, best,
                      tour, excluded, -1, -1, "Tour found!");
            found_tour = true;
        }
    }
    if (!found_tour) {
        push_step(build_full(cmat, row_cities, col_cities), bound, best,
                  included, excluded, -1, -1,
                  "Cannot complete tour (edge missing)");
    }
    return;
}
```

```
// Find branching edge (max penalty zero)
int best_i = -1, best_j = -1, max_penalty = -1;
for (int i = 0; i < m; i++) {
   for (int j = 0; j < m; j++) {
      if (cmat[i][j] != 0) continue;
      int row_min = BNB_INF;
      for (int k = 0; k < m; k++)
         if (k != j && cmat[i][k] < row_min) row_min = cmat[i][k];
      int col_min = BNB_INF;
      for (int k = 0; k < m; k++)
         if (k != i && cmat[k][j] < col_min) col_min = cmat[k][j];
      int penalty = (row_min < BNB_INF / 2 ? row_min : 0) +
                    (col_min < BNB_INF / 2 ? col_min : 0);
      if (penalty > max_penalty) {
         max_penalty = penalty;
         best_i = i;
         best_j = j;
      }
   }
}

if (best_i < 0 || best_j < 0) return; // no zero

int orig_i = row_cities[best_i];
int orig_j = col_cities[best_j];

push_step(build_full(cmat, row_cities, col_cities), bound, best,
        included, excluded, orig_i, orig_j,
        "Branch on (" + std::to_string(orig_i + 1) + "," +
           std::to_string(orig_j + 1) + ") penalty=" +
           std::to_string(max_penalty));

// Exclude branch: forbid edge (orig_i, orig_j)
{
   auto exc_mat = cmat;
   exc_mat[best_i][best_j] = BNB_INF;
   auto exc_exc = excluded;
   exc_exc.emplace_back(orig_i, orig_j);
   push_step(build_full(exc_mat, row_cities, col_cities), bound, best,
           included, exc_exc, orig_i, orig_j,
           "Exclude (" + std::to_string(orig_i + 1) + "," +
              std::to_string(orig_j + 1) + ")");
   bnb_rec(exc_mat, bound, included, exc_exc, row_cities, col_cities);
}
```

```
// Include branch: add edge, remove row/col, forbid reverse
{
  // Correct reverse edge: from orig_j to orig_i
  int rev_i = -1, rev_j = -1;
  for (int k = 0; k < m; k++) {
    if (row_cities[k] == orig_j) rev_i = k;
    if (col_cities[k] == orig_i) rev_j = k;
  }
  if (rev_i >= 0 && rev_j >= 0)
    cmat[rev_i][rev_j] = BNB_INF;

  std::vector<std::vector<int>> inc_mat(m - 1, std::vector<int>(m - 1));
  std::vector<int> new_rows, new_cols;

  for (int i = 0; i < m; i++) {
    if (i == best_i) continue;
    new_rows.push_back(row_cities[i]);
    int ri = (i > best_i) ? i - 1 : i;
    for (int j = 0; j < m; j++) {
      if (j == best_j) continue;
      inc_mat[ri][(j > best_j) ? j - 1 : j] = cmat[i][j];
    }
  }
  for (int j = 0; j < m; j++)
    if (j != best_j) new_cols.push_back(col_cities[j]);

  auto inc_inc = included;
  inc_inc.emplace_back(orig_i, orig_j);

  // Subtour prevention: forbid edge (tail, head)
  int sub_tail = orig_j;
  for (bool cont = true; cont;) {
    cont = false;
    for (auto &e : inc_inc)
      if (e.first == sub_tail) { sub_tail = e.second; cont = true; break; }
  }
  int sub_head = orig_i;
  for (bool cont = true; cont;) {
    cont = false;
    for (auto &e : inc_inc)
      if (e.second == sub_head) { sub_head = e.first; cont = true; break; }
  }
  if (!(sub_tail == orig_j && sub_head == orig_i)) {
    int fi = -1, fj = -1;
    for (int k = 0; k < m - 1; k++) {
```

```
                if (new_rows[k] == sub_tail) fi = k;
                if (new_cols[k] == sub_head) fj = k;
            }
            if (fi >= 0 && fj >= 0)
                inc_mat[fi][fj] = BNB_INF;
        }

        push_step(build_full(inc_mat, new_rows, new_cols), bound, best,
                  inc_inc, excluded, orig_i, orig_j,
                  "Include (" + std::to_string(orig_i + 1) + "," +
                    std::to_string(orig_j + 1) + ")");
        bnb_rec(inc_mat, bound, inc_inc, excluded, new_rows, new_cols);
    }
  };

  std::vector<int> init_cities(n);
  for (int i = 0; i < n; i++) init_cities[i] = i;

  bnb_rec(mat, 0, {}, {}, init_cities, init_cities);

  if (best < BNB_INF / 2) {
    push_step(mat, best, best, best_path, {}, -1, -1,
              "Optimal tour length: " + std::to_string(best));
  } else {
    push_step(mat, 0, BNB_INF, {}, {}, -1, -1, "No tour exists");
  }
}
```

КОД Graph.h

```
//
// Created by localuser on 5/13/26.
//

#include "MatrixEditor.h"

#include <QHeaderView>
#include <QVBoxLayout>
#include <QHBoxLayout>

MatrixEditor::MatrixEditor(const Graph &graph, QWidget *parent)
  : QDialog(parent), graph(graph) {
  setWindowTitle("Edit matrix");

  table = new QTableWidget(this);
```

```
    table->setEditTriggers(QAbstractItemView::DoubleClicked
                           | QAbstractItemView::SelectedClicked
                           | QAbstractItemView::EditKeyPressed);
    table->horizontalHeader()->setSectionResizeMode(QHeaderView::Stretch);
    table->verticalHeader()->setSectionResizeMode(QHeaderView::Stretch);

    add_vertex = new QPushButton("Add vertex", this);
    remove_vertex = new QPushButton("Remove vertex", this);
    buttons = new QDialogButtonBox(QDialogButtonBox::Ok | QDialogButtonBox::Cancel, this);

    auto *buttons_layout = new QHBoxLayout;
    buttons_layout->addWidget(add_vertex);
    buttons_layout->addWidget(remove_vertex);
    buttons_layout->addWidget(buttons);

    auto *main_layout = new QVBoxLayout(this);
    main_layout->addWidget(table);
    main_layout->addLayout(buttons_layout);
    setLayout(main_layout);

    load_from_graph();

    connect(add_vertex, &QPushButton::clicked, this, &MatrixEditor::add_vertex_row_col);
    connect(remove_vertex, &QPushButton::clicked, this,
&MatrixEditor::remove_vertex_row_col);
    connect(buttons, &QDialogButtonBox::accepted, this, &QDialog::accept);
    connect(buttons, &QDialogButtonBox::rejected, this, &QDialog::reject);
}

void MatrixEditor::load_from_graph() {
    const auto &matrix = graph.get_matrix();
    int n = matrix.size();

    table->setRowCount(n);
    table->setColumnCount(n);

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (matrix[i][j] == nullptr) {
                table->setItem(i, j, new QTableWidgetItem(""));
            } else {
                table->setItem(i, j, new QTableWidgetItem(QString::number(*matrix[i][j])));
            }
        }
    }
```

```
}

std::vector<std::vector<int *> > MatrixEditor::build_matrix_from_table() const {
    int n = table->rowCount();
    std::vector<std::vector<int *> > matrix(n, std::vector<int *>(n, nullptr));

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            QTableWidgetItem *item = table->item(i, j);
            if (!item) continue;
            QString text = item->text().trimmed();
            if (text.isEmpty()) continue;
            bool ok = false;
            int value = text.toInt(&ok);
            if (!ok) continue;
            matrix[i][j] = new int(value);
        }
    }

    return matrix;
}

void MatrixEditor::add_vertex_row_col() {
    int n = table->rowCount();
    table->setRowCount(n + 1);
    table->setColumnCount(n + 1);

    for (int i = 0; i < n + 1; i++) {
        for (int j = 0; j < n + 1; j++) {
            if (table->item(i, j)) continue;
            table->setItem(i, j, new QTableWidgetItem(""));
        }
    }
}

void MatrixEditor::remove_vertex_row_col() {
    int n = table->rowCount();
    table->setRowCount(n - 1);
    table->setColumnCount(n - 1);

    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - 1; j++) {
            if (table->item(i, j)) continue;
            table->setItem(i, j, new QTableWidgetItem(""));
        }
```

```
    }
}

std::vector<std::vector<int *> > MatrixEditor::build_matrix() const {
    return build_matrix_from_table();
}
```

КОД MatrixEditor.cpp

```
//
// Created by localuser on 5/12/26.
//

#ifndef QTGRAPHVISUALIZATION_GRAPH_H
#define QTGRAPHVISUALIZATION_GRAPH_H
#include <cstdlib>
#include <queue>
#include <vector>

class Graph {
    std::vector<std::vector<int *> > matrix;

public:
    Graph() = default;
    Graph(const Graph &other);
    ~Graph();
    Graph &operator=(const Graph &other);
    void clear_matrix();
    const std::vector<std::vector<int *> > &get_matrix() const;
    void set_matrix(const std::vector<std::vector<int *> > &new_matrix);

    bool is_visual_tree();

    void generateRandom(int n, int maxWeight = 10, double edgeProbability = 0.5);
    void generateTree(int n, int maxWeight = 10);

    int size();
    bool has_edge(int i, int j);
    int *get_weight(int i, int j);

    void dfs(uint n, std::vector<bool>& visited);
};


#endif //QTGRAPHVISUALIZATION_GRAPH_H
```

КОД Graph.cpp

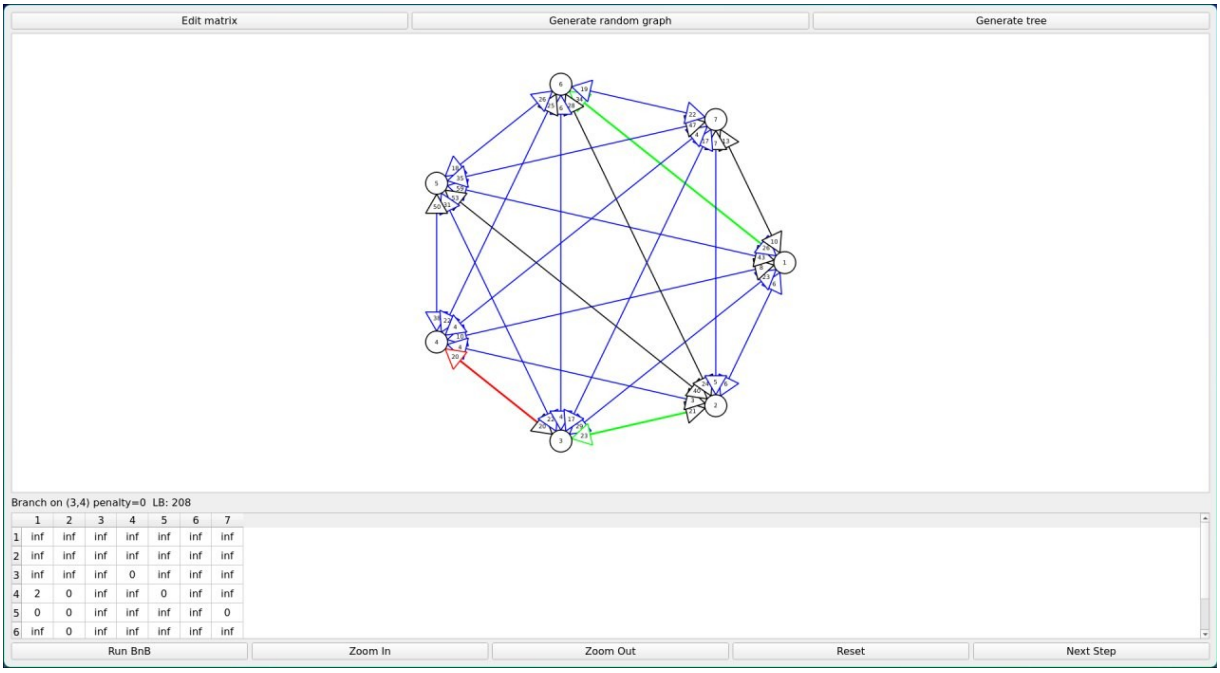
```cpp
#include "Graph.h"

#include <random>

static std::mt19937 rng(std::random_device{}());

Graph::Graph(const Graph &other) {
    set_matrix(other.matrix);
}

Graph::~Graph() {
    clear_matrix();
}

void Graph::clear_matrix() {
    for (int i = 0; i < matrix.size(); i++)
        for (int j = 0; j < matrix[i].size(); j++)
            if (matrix[i][j] != nullptr)
                delete matrix[i][j];

    matrix.clear();
}

Graph &Graph::operator=(const Graph &other) {
    if (this == &other)
        return *this;

    set_matrix(other.matrix);

    return *this;
}

const std::vector<std::vector<int *> > &Graph::get_matrix() const {
    return matrix;
}

void Graph::set_matrix(const std::vector<std::vector<int *> > &new_matrix) {
    clear_matrix();

    matrix.resize(new_matrix.size());
    for (int i = 0; i < new_matrix.size(); i++)
```

```
        for (int j = 0; j < new_matrix[i].size(); j++)
            if (new_matrix[i][j] == nullptr)
                matrix[i].push_back(nullptr);
            else
                matrix[i].push_back(new int(*new_matrix[i][j]));
}

bool Graph::is_visual_tree() {
    int n = matrix.size();
    if (n == 0) return false;

    int edgeCount = 0;
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++)
            if (matrix[i][j] != nullptr || matrix[j][i] != nullptr)
                edgeCount++;

    if (edgeCount != n - 1) return false;

    std::vector<bool> visited(n, false);
    std::queue<int> q;
    q.push(0);
    visited[0] = true;
    int visitedCount = 1;

    while (!q.empty()) {
        int cur = q.front();
        q.pop();

        for (int next = 0; next < n; next++) {
            if (!visited[next] &&
                (matrix[cur][next] != nullptr || matrix[next][cur] != nullptr)) {
                visited[next] = true;
                visitedCount++;
                q.push(next);
            }
        }
    }

    return visitedCount == n;
}

void Graph::generateRandom(int n, int maxWeight, double edgeProbability) {
    for (auto &row: matrix)
        for (auto *p: row)
```

```
            delete p;

    matrix.assign(n, std::vector<int *>(n, nullptr));

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i == j) continue;
            std::bernoulli_distribution edge_dist(edgeProbability);
            if (edge_dist(rng)) {
                std::uniform_int_distribution<int> weight_dist(1, maxWeight);
                matrix[i][j] = new int(weight_dist(rng));
            }
        }
    }
}

void Graph::generateTree(int n, int maxWeight) {
    for (auto &row: matrix)
        for (auto *p: row)
            delete p;

    matrix.assign(n, std::vector<int *>(n, nullptr));

    std::vector<int> perm(n);
    for (int i = 0; i < n; i++) perm[i] = i;
    for (int i = n - 1; i > 0; i--) {
        std::uniform_int_distribution<int> idx_dist(0, i);
        int j = idx_dist(rng);
        std::swap(perm[i], perm[j]);
    }

    for (int i = 1; i < n; i++) {
        int parent = perm[std::rand() % i];
        int child = perm[i];
        std::uniform_int_distribution<int> weight_dist(1, maxWeight);
        matrix[parent][child] = new int(weight_dist(rng));
    }
}

int Graph::size() {
    return matrix.size();
}

bool Graph::has_edge(int i, int j) {
    return matrix[i][j];
```

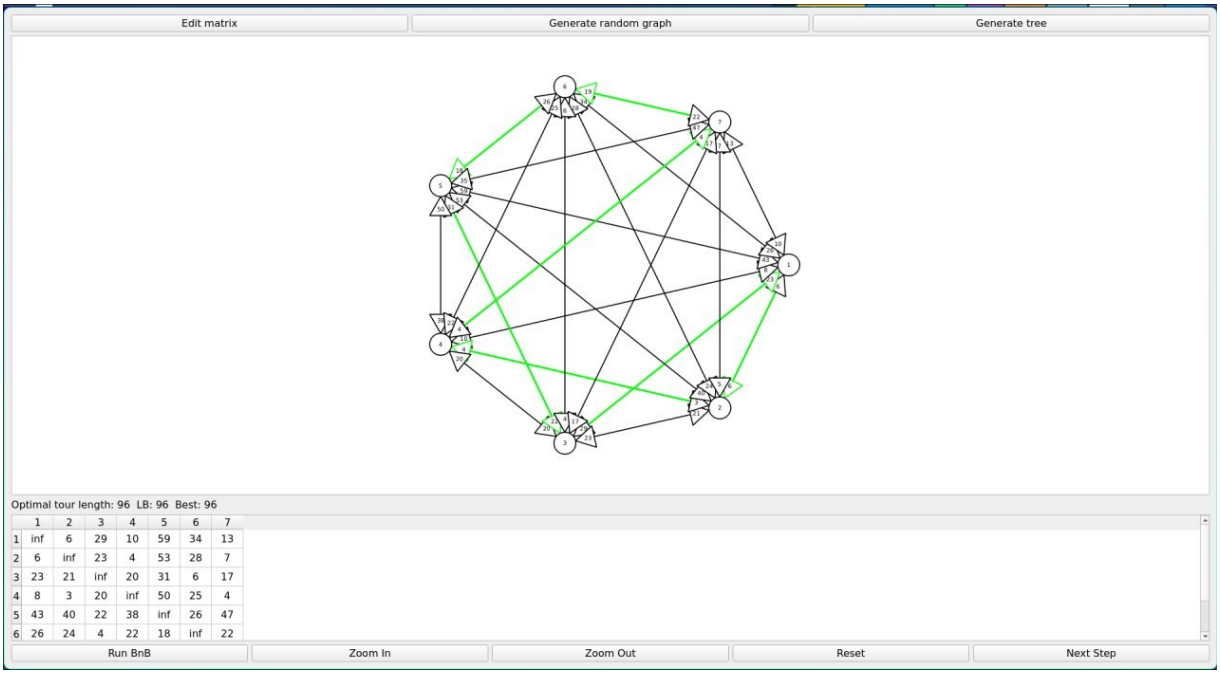
```
}

int *Graph::get_weight(int i, int j) {
   return matrix[i][j];
}

void Graph::dfs(uint n, std::vector<bool> &visited) {
   visited[n] = true;
   for (auto& i : matrix[n]) {
      if (i != nullptr && !visited[*i]) {
         dfs(*i, visited);
      }
   }
}
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ



	1	2	3	4	5	6	7
1	inf	inf	inf	inf	inf	inf	inf
2	inf	inf	inf	inf	inf	inf	inf
3	inf	inf	inf	0	inf	inf	inf
4	2	0	inf	inf	0	inf	inf
5	0	0	inf	inf	inf	inf	0
6	inf	0	inf	inf	inf	inf	inf

Пример шага алгоритма



	1	2	3	4	5	6	7
1	inf	6	29	10	59	34	13
2	6	inf	23	4	53	28	7
3	23	21	inf	20	31	6	17
4	8	3	20	inf	50	25	4
5	43	40	22	38	inf	26	47
6	26	24	4	22	18	inf	22

Итоговый цикл: 3→1→2→4→7→6→5→3 с ценой 96